

TUGAS KECIL 3 STRATEGI ALGORITMA

**Penyelesaian Permainan Ice Sliding Puzzle Solver
dengan Algoritma UCS, GBFS, A*, dan Iterative Deepening A***



Disusun Oleh :

Nathan Adhika Santosa - 13524041

Valentino Daniel Kusumo - 13524104

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1.....	3
1. Permainan Ice Sliding Puzzle Solver.....	3
2. Elemen Papan Permainan.....	3
3. Algoritma UCS.....	4
4. Algoritma Greedy Best First Search (GBFS).....	6
5. Algoritma A*.....	9
6. Algoritma Iterative Deepening A*.....	13
7. Analisis algoritma pathfinding.....	16
BAB 2.....	18
1. Link Repository Program.....	18
2. Source Code Program.....	18
3. Implementasi GUI.....	30
BAB 3.....	34
1. Kasus Uji 1 - valid.....	34
2. Kasus Uji 2 - invalid (no N and M).....	36
3. Kasus Uji 3 - invalid (suatu baris jumlah kolomnya != M).....	37
4. Kasus Uji 4 - valid.....	38
5. Kasus Uji 5 - No solution (lewati lava).....	40
6. Kasus Uji 6 - valid.....	41
7. Analisis Hasil Percobaan Algoritma.....	43
BAB IV.....	45

BAB 1

DESKRIPSI PERMAINAN DAN ALGORITMA

1. Permainan Ice Sliding Puzzle Solver

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar (*goal*) di atas permukaan es yang licin. Pemain hanya dapat bergerak secara horizontal atau vertikal, tetapi karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan.

Tujuan utama permainan Ice Sliding Puzzle adalah memindahkan karakter dari posisi awal (Z) menuju titik tujuan (O) dengan kondisi kemenangan sebagai berikut:

1. Pemain harus berhenti tepat di tile O, tidak cukup hanya melewatinya karena aktor terus meluncur hingga terhalang sesuatu.
2. Semua checkpoint angka (0–9) harus sudah dilalui sesuai urutan.
3. Tanpa menyentuh lava (L) dan tanpa keluar dari batas papan sepanjang perjalanan.

2. Elemen Papan Permainan

Simbol	Nama	Keterangan
Z	Pemain	Posisi awal karakter yang dikendalikan
O	Titik Tujuan	Posisi yang harus dicapai pemain
*	Path	Tile kosong yang dapat dilewati secara bebas
X	Batu/Rintangan	Menghalangi pergerakan pemain, berhenti tepat sebelum batu
L	Lava	Menyebabkan <i>game over</i> jika dilewati meskipun pemain tidak berhenti di lava
0 - 9	Checkpoint berurutan	Tile angka yang harus dilewati sesuai urutan

3. Algoritma UCS

UCS adalah algoritma pencarian yang selalu menelusuri node dengan total cost terendah terlebih dahulu. UCS menjamin solusi yang ditemukan adalah solusi optimal karena selalu mengambil rute yang biayanya terendah. Dalam konteks permainan Ice Sliding Puzzle, cost yang diminimalkan adalah total cost traversal tile yang dilalui pemain sepanjang seluruh gerakan *sliding*.

Komponen utama

- **Priority Queue:** Menyimpan node yang akan dieksplorasi, diurutkan berdasarkan $g(n)$ terkecil
- **Visited Set:** Mencatat state yang sudah dieksplorasi agar tidak diproses ulang
- **State:** Kombinasi posisi pemain dan checkpoint berikutnya yang harus dilewati (NextNum)
- **$g(n)$:** Total cost dari start hingga node saat ini

Langkah - Langkah Algoritma

1. Inisialisasi

- Buat node awal dari posisi start (Z) dengan $cost = 0$, $NextNum = 0$, dan tanpa parent.
- Masukkan node awal ke dalam Priority Queue.
- Siapkan visited set yang kosong.

2. Ambil Node Terbaik

- Ambil node dengan cost terkecil dari Priority Queue.
- Tambahkan iterasi counter.

3. Cek Visited

- Jika state dari node tersebut sudah pernah dikunjungi, *skip* dan kembali ke langkah 2.
- Jika belum, tandai state tersebut sebagai visited.

4. Cek Kondisi Goal

- Periksa apakah posisi pemain saat ini adalah tile tujuan (O) dan semua checkpoint angka sudah dilewati.
- Jika ya, *return* node ini sebagai solusi.

5. Ekspansi Node

- Untuk setiap arah (Atas, Bawah, Kiri, Kanan):
 - Lakukan *sliding* ke arah tersebut.
 - Jika gerakan valid:
 - Hitung $\text{newCost} = \text{cost sekarang} + \text{moveCost}$.
 - Buat node baru dengan state hasil sliding, cost baru, dan parent node saat ini.
 - Masukkan node baru ke Priority Queue.
 - Kembali ke langkah 2.

6. Tidak ada Solusi

- Jika Priority Queue kosong dan goal belum ditemukan, *return* nil (tidak ada solusi).

Berikut adalah *pseudocode* untuk algoritma UCS :

```
function UCS(board: Board) -> Node, integer
  pq <- PriorityQueue()
  visited <- HashSet()
  iter <- 0

  startNode <- Node(
    state  = State{pos = board.start, nextNum
    = 0},
    cost   = 0,
    depth  = 0,
    parent = nil,
    dir    = Nil
  )
  pq.push(startNode)

  while not pq.isEmpty()
    curr <- pq.pop()
    iter <- iter + 1

    if visited[curr.state] then
      continue
```

```

visited[curr.state] <- true

if IsGoal(curr.state.pos, board) and
curr.state.nextNum =
len(board.checkpoint) then
    -> curr, iter

for dir in [Up, Down, Left, Right]
    nextState, moveCost, valid <-
        slide(curr.state, dir, board)

    if valid then
        nextNode <- Node(
            state = nextState,
            cost = curr.cost +
                moveCost,
            depth = curr.depth
                + 1,
            parent = curr,
            dir = dir
        )
        pq.push(nextNode)

-> nil, iter

```

4. Algoritma Greedy Best First Search (GBFS)

Greedy Best First Search (GBFS) adalah algoritma pencarian rute yang selalu menelusuri node dengan nilai heuristic terkecil terlebih dahulu. GBFS berusaha mencapai *goal* secepat mungkin dengan memilih node yang diperkirakan paling dekat menuju tujuan. Dalam konteks permainan Ice Sliding Puzzle, heuristic yang digunakan adalah jarak Manhattan antara posisi pemain saat ini dengan target berikutnya, yaitu checkpoint selanjutnya atau goal.

Komponen Utama :

- **Priority Queue:** Menyimpan node yang akan dieksplorasi, diurutkan berdasarkan nilai heuristic $h(n)$ terkecil
- **Visited Set:** Mencatat state yang sudah dieksplorasi agar tidak diproses ulang
- **State:** Kombinasi posisi pemain dan checkpoint berikutnya yang harus dilewati (NextNum)
- **Heuristic $h(n)$:** Estimasi jarak dari node saat ini menuju target berikutnya

Langka-langkah Algoritma :

1. Inisialisasi

- Buat node awal dari posisi start (Z) dengan heuristic sebagai jarak Manhattan menuju target pertama, NextNum = 0, tanpa parent
- Masukkan node awal ke dalam Priority Queue.
- Siapkan visited set dan stateCost map.

2. Ambil Node Terbaik

- Ambil node dengan heuristic terkecil dari Priority Queue.
- Tambahkan iterasi counter.

3. Cek Kondisi Goal

- Periksa apakah posisi pemain saat ini adalah tile tujuan dan semua checkpoint angka sudah dilewati.
- Jika ya, *return* node ini sebagai solusi.

4. Ekspansi Node

- Untuk setiap arah (Atas, Bawah, Kiri, Kanan):
 - Lakukan sliding ke arah tersebut.
 - Jika gerakan valid dan state belum pernah dikunjungi:
 - Hitung total cost perjalanan.
 - Hitung heuristic baru menggunakan jarak Manhattan menuju target berikutnya.
 - Buat node baru dengan state hasil sliding, heuristic baru, dan parent node saat ini.
 - Masukkan node baru ke Priority Queue.
 - Kembali ke langkah 2.

5. Tidak ada Solusi

- Jika Priority Queue kosong dan goal belum ditemukan, return nil (tidak ada solusi).

Fungsi heuristic yang digunakan adalah:

$$h(n) = |x1 - x2| + |y1 - y2|$$

dengan :

- $(x1,y1)$ = posisi pemain saat ini
- $(x2,y2)$ = posisi target berikutnya

Berikut adalah pseudocode GBFS :

```
function GreedyBestFirstSearch (board: Board) -> Array  
of Point, integer, integer  
pathTaken <- Array()  
totalCost <- 0  
iter <- 0  
  
pq <- PriorityQueue()  
visited <- Map()  
stateCost <- Map()  
  
startState <- State{pos = board.start, nextNum = 0}  
stateCost[startState] <- 0  
  
startNode <- Node (  
    state = startState,  
    cost = CalculateHeuristic(board, startState),  
    depth = 0,  
    parent = nil,  
    dir = Nil  
)  
  
pq.push(startNode)  
visited[startState] <- true  
finalNode <- nil  
  
while not pq.isEmpty()  
    curr <- pq.pop()  
    iter <- iter + 1  
  
    if IsGoal(curr.state.pos, board) and  
    curr.state.nextNum = len(board.checkpoint) then  
        finalNode <- curr  
        break  
  
    for dir in [Up, Down, Left, Right]  
        nextState, moveCost, moved <- slide(curr.state,
```

```

dir, board)

    if moved and not visited[nextState] then
        visited[nextState] <- true
        stateCost[nextState] <-
            stateCost[curr.state] + moveCost

        neighborNode <- Node (
            state = nextState,
            cost = CalculateHeuristic(board,
                nextState),
            depth = curr.depth + 1,
            parent = curr,
            dir = dir
        )

        pq.push(neighborNode)

if finalNode = nil then
    -> pathTaken, 0, iter

currNode <- finalNode
while currNode != nil
    pathTaken.append(currNode.state.pos)
    currNode <- currNode.parent

Reverse(pathTaken)
totalCost <- stateCost[finalNode.state]

-> pathTaken, totalCost, iter

```

5. Algoritma A*

A* Search merupakan algoritma pencarian heuristic yang menggabungkan keunggulan Uniform Cost Search (UCS) dan Greedy Best First Search (GBFS). Algoritma ini memilih node berdasarkan total estimasi biaya terkecil menuju goal, yaitu hasil penjumlahan antara biaya perjalanan yang sudah ditempuh dan estimasi biaya menuju tujuan. Dalam permainan Ice Sliding Puzzle, heuristic yang digunakan adalah jarak Manhattan (metode penghitungan jarak antara dua titik dengan menjumlahkan selisih mutlak dari koordinat-koordinatnya) antara posisi pemain saat ini dengan target berikutnya, sedangkan biaya perjalanan dihitung berdasarkan total cost tile yang telah dilewati.

Komponen utama :

- **Priority Queue:** Menyimpan node yang akan dieksplorasi berdasarkan nilai $f(n)$ terkecil
- **Visited Set:** Menyimpan state yang sudah pernah dikunjungi
- **StateCost Map:** Menyimpan biaya minimum menuju suatu state
- **State:** Kombinasi posisi pemain dan checkpoint berikutnya yang harus dilewati
- **Heuristic $h(n)$:** Estimasi jarak Manhattan menuju target berikutnya

Langkah-Langkah Algoritma :

1. Inisialisasi

- Buat node awal dari posisi start (Z) dengan:
 - $g(n) = 0$
 - Heuristik menggunakan jarak manhattan
 - $\text{Cost node} = f(n) + h(n)$
 - $\text{NextNum} = 0$
 - Tanpa node parent
- Masukkan node awal ke queue
- Siapkan visited set dan stateCost map

2. Ambil Node Terbaik

- Ambil node dengan nilai $f(n)$ terkecil dari Priority queue
- Tambahkan iterasi counter

3. Cek kondisi goal

- Periksa apakah posisi pemain saat ini adalah tile tujuan dan semua checkpoint angka sudah dilewati
- Jika iya, return node ini sebagai solusi

4. Ekspansi Goal

- Untuk setiap arah :
 - Lakukan sliding ke arah tersebut
 - Jika gerakan valid, hitung newCost sebagai :

$$g(n_{\text{baru}}) = g(n_{\text{sekarang}}) + \text{moveCost}$$

- Jika state belum pernah dikunjungi atau ditemukan jalur dengan cost lebih kecil:
 - Update stateCost
 - Hitung heuristic baru menggunakan jarak manhattan menuju tujuan berikutnya.
 - Hitung $f(n)$
 - Buat node baru dengan parent node saat ini
 - Masukkan node baru ke priority
- Kembali ke langkah 2.

Fungsi evaluasi algoritma A* adalah

$$f(n) = g(n) + h(n)$$

dengan

- $g(n)$ = total cost perjalanan dari start menuju node n
- $h(n)$ = estimasi biaya dari node n menuju target (jarak manhattan)

Berikut adalah pseudocode untuk algoritma A*

```
function AStarSearch (board: Board) -> Array of Point,
integer, integer
pathTaken <- Array()
totalCost <- 0
iter <- 0

pq <- PriorityQueue()
visited <- Map()
stateCost <- Map()

startState <- State{pos = board.start, nextNum = 0}
stateCost[startState] <- 0

startNode <- Node (
  state = startState,
  cost = stateCost[startState] +
    CalculateHeuristic(board, startState),
  depth = 0,
  parent = nil,
  dir = Nil
)

pq.push(startNode)
```

```

visited[startState] <- true
finalNode <- nil

while not pq.isEmpty()
  curr <- pq.pop()
  iter <- iter + 1

  if IsGoal(curr.state.pos, board) and
  curr.state.nextNum = len(board.checkpoint) then
    finalNode <- curr
    break

  for dir in [Up, Down, Left, Right]
    nextState, moveCost, moved <- slide(curr.state,
    dir, board)

    if moved then
      newCost <- stateCost[curr.state] + moveCost

      if not visited[nextState] or newCost <
      stateCost[nextState] then
        visited[nextState] <- true
        stateCost[nextState] <- newCost

        neighborNode <- Node (
          state = nextState,
          cost = newCost +
          CalculateHeuristic(board,
          nextState),
          depth = curr.depth + 1,
          parent = curr,
          dir = dir
        )

        pq.push(neighborNode)

if finalNode = nil then
  -> pathTaken, 0, iter

currNode <- finalNode
while currNode != nil
  pathTaken.append(currNode.state.pos)
  currNode <- currNode.parent

Reverse(pathTaken)
totalCost <- stateCost[finalNode.state]

-> pathTaken, totalCost, iter

```

6. Algoritma Iterative Deepening A*

Iterative Deepening A* (IDA*) merupakan algoritma pencarian heuristic yang menggabungkan konsep Depth First Search (DFS) dengan evaluasi cost seperti pada algoritma A*. Berbeda dengan A* yang menggunakan priority queue, IDA* melakukan pencarian berulang dengan meningkatkan threshold secara bertahap. Pendekatan ini membuat IDA* lebih hemat memori dibandingkan A* karena hanya menyimpan jalur pencarian aktif seperti DFS.

Komponen Utama

- **Threshold:** Batas maksimum nilai $f(n)$ yang boleh dieksplorasi
- **Path Map:** Menyimpan state yang sedang berada pada jalur DFS untuk mencegah siklus
- **Recursive DFS :** Digunakan untuk melakukan eksplorasi secara depth-first
- **Heuristik $h(n)$:** Estimasi jarak manhattan menuju target berikut

Langkah-Langkah Algoritma

1. Inisialisasi

- Buat node awal dari posisi awal dengan:
 - $g(n) = 0$
 - Heuristik menggunakan jarak manhattan
 - $\text{CostNode} : g(n) + h(n)$
 - Tanpa Parent
- Threshold awal diisi dengan heuristik node awal

2. Depth-Limited Search

- Lakukan DFS menggunakan fungsi rekursif `idaStarDFS`
- Hitung $f(n)$
- Jika $f(n) > \text{threshold}$, maka node tidak dieksplorasi dan $f(n)$ dikembalikan sebagai kandidat threshold berikut

3. Cek kondisi goal

- Jika posisi pemain berada pada goal dan seluruh checkpoint telah dilewati, maka solusi ditemukan dan pencarian berhenti

4. Ekspansi Node

- Untuk setiap arah:
 - Lakukan sliding ke arah tersebut
 - Jika state valid dan belum berada pada path saat ini:
 - Tambahkan state ke pathMap
 - Hitung $g(n_{\text{baru}}) = g(n_{\text{sekarang}}) + \text{moveCost}$
 - Hitung $f(n_{\text{baru}}) = g(n_{\text{baru}}) + h(n_{\text{baru}})$
 - Lakukan DFS rekursif terhadap node baru
 - Setelah selesai, hapus state dari pathmap

5. Update Threshold

- Jika goal belum ditemukan, threshold diperbarui menggunakan nilai minimum $f(n)$ yang melebihi threshold sebelumnya.
- Pencarian kemudian diulang kembali dari root.

6. Tidak ada solusi

- Jika tidak ditemukan threshold baru, maka puzzle tidak memiliki solusi.

Berikut adalah pseudocode untuk algoritma IDA* :

```
Constant FOUND <- -1
Constant Infinity <- MaxInt32

function idaStarDFS (board: Board, currNode: Node, g:
integer, threshold: integer, pathMap: Map, iter:
reference to integer) -> integer, Node
    iter <- iter + 1
    h <- CalculateHeuristic(board, currNode.state)
    f <- g + h

    if f > threshold then
        -> f, nil

    if IsGoal(currNode.state.pos, board) and
currNode.state.nextNum = len(board.checkpoint) then
        -> FOUND, currNode

    min <- Infinity
```

```

for dir in [Up, Down, Left, Right]
  nextState, moveCost, moved <-
    slide(currNode.state, dir, board)

  if moved and not pathMap[nextState] then
    pathMap[nextState] <- true
    newG <- g + moveCost

    neighborNode <- Node (
      state = nextState,
      cost = newG + CalculateHeuristic(board,
        nextState),
      depth = currNode.depth + 1,
      parent = currNode,
      dir = dir
    )

    t, resultNode <- idaStarDFS(board,
      neighborNode, newG, threshold, pathMap,
      iter)

    if t = FOUND then
      -> FOUND, resultNode

    if t < min then
      min <- t

    delete pathMap[nextState]

-> min, nil

```

```

function IDAStarSearch (board: Board) -> Array of Point,
integer, integer
  startState <- State{pos = board.start, nextNum = 0}
  iter <- 0

  startNode <- Node (
    state = startState,
    cost = CalculateHeuristic(board, startState),
    depth = 0,
    parent = nil,
    dir = Nil
  )

  threshold <- CalculateHeuristic(board, startState)

  while true
    pathMap <- Map()
    pathMap[startState] <- true

```

```

t, finalNode <- idaStarDFS(board, startNode, 0,
threshold, pathMap, iter)

if t = FOUND then
  pathTaken <- Array()
  curr <- finalNode

  while curr != nil
    pathTaken.append(curr.state.pos)
    curr <- curr.parent

  Reverse(pathTaken)
  -> pathTaken, finalNode.cost, iter

if t = Infinity then
  -> Array(), 0, iter

threshold <- t

```

7. Analisis algoritma *pathfinding*

Algoritma	Definisi fungsi	Admissible?	Perbandingan Algoritma	Iterasi
UCS	$f(n) = g(n)$, dimana $g(n)$ adalah total biaya selama menelusuri	Tidak, karena bukan informed search	Menemukan path paling optimal, tapi lebih lambat dan menggunakan memori paling banyak, sebab tidak menggunakan heuristik untuk melakukan penelusuran.	Iterasi terjadi setiap kali program mengambil pop sebuah simpul dari Priority Queue untuk dievaluasi. Dalam setiap iterasinya, UCS akan selalu memilih simpul dengan $g(n)$ paling rendah. Setelah simpul tersebut diambil, algoritma akan mengekspansi simpul itu dengan mencoba semua arah pergerakan yang valid, lalu memasukkan status-status posisi yang baru ke dalam queue. Siklus ini diulang secara terus-menerus hingga mendapatkan simpul tujuan.
GBFS	$f(n) = h(n)$, dimana $h(n)$ adalah nilai jarak manhattan terendah dari awal ke tujuan	Tidak, karena algoritma hanya memperhatikan nilai jarak manhattan, bukan nilai biaya total.	Paling cepat, tetapi tidak menghasilkan rute paling optimal karena algoritma tidak memperhatikan jumlah biaya total penelusuran	Mirip dengan UCS, tapi GBFS memilih simpul dengan $h(n)$ paling rendah.

A*	$f(n) = g(n) + h(n)$, dimana $g(n)$ dan $h(n)$ sama persis dengan definisi sebelumnya	Iya, karena A* memperhatikan biaya total untuk menuju ke tujuan, sehingga biaya menuju tujuan lebih kecil sama dengan biaya aktualnya.	Paling cepat dan menghasilkan rute paling optimal, sebab mempertimbangkan jumlah biaya total dan nilai heuristik (<i>jarak manhattan</i>) terendah. Namun, menggunakan banyak memori karena jalur yang tidak ditelusuri tetap disimpan.	Mirip dengan UCS, tapi A* memilih simpul dengan $g(n)$ dan $h(n)$ paling rendah.
IDA*	$f(n) = g(n) + h(n)$, dimana $g(n)$ dan $h(n)$ sama persis dengan definisi sebelumnya	Iya, karena IDA* juga memperhatikan biaya total yang terletak di threshold untuk menuju ke tujuan, sehingga biaya menuju tujuan $\leq$ biaya aktualnya.	Optimal dan hemat memori, sebab menggunakan fungsi-fungsi A* dengan penelusuran DFS. Tapi lebih lambat dibandingkan dengan A* karena pencarian harus ulang dari awal jika threshold menambah.	Iterasi dikendalikan oleh threshold. Iterasi utama terjadi setiap kali algoritma gagal menemukan titik tujuan pada threshold saat ini, sehingga ia harus menaikkan nilai threshold tersebut dan mengulang kembali seluruh pencarian dari titik awal menggunakan pendekatan DFS. Di dalam pencarian tersebut, fungsi akan terus menelusuri rute secara mendalam selama nilai $f(n)$, tidak melebihi threshold. Jika melampaui, maka dilakukan backtracking.

BAB 2

SOURCE CODE PROGRAM DAN GUI

1. Link Repository Program

https://github.com/ValentinoDan/Tucil3_13524041_13524104

2. Source Code Program

Struktur program :

```
|—bin
|—doc
|—result
|—src
|   |—algorithm
|   |—filehandler
|   |—font
|   |—gui
|   |—puzzle
|   |—main.go
|—test
|—README.md
|—go.mod
|—go.sum
```

board.go

```
package puzzle

import "fmt"

type Board struct {
    N, M      int
    Grid      [][]rune
    Cost      [][]int
    Start, Goal Point
```

```

    Checkpoint map[int]Point
    Lava       []Point
}

type Point struct {
    Row, Col int
}

// Checker functions
func IsWall(p Point, board *Board) bool {
    return (board.Grid[p.Row][p.Col] == 'X')
}

func IsLava(p Point, board *Board) bool {
    for _, l := range board.Lava {
        if l.Row == p.Row && l.Col == p.Col {
            return true
        }
    }
    return false
}

func IsGoal(p Point, board *Board) bool {
    return (board.Goal.Row == p.Row && board.Goal.Col == p.Col)
}

func IsInBounds(p Point, board *Board) bool {
    return (p.Row >= 0 && p.Row < board.N && p.Col >= 0 && p.Col
            < board.M)
}

func GetCost(p Point, board *Board) int {
    return board.Cost[p.Row][p.Col]
}

```

helper.go

```

package algorithm

import (
    "iceSlidingPuzzle/src/puzzle"
)

```

```

// mutlak x
func Abs(x int) int {
    if x < 0 {
        return -x
    }
    return x
}

// slide until hits wall / stop
func slide(startState puzzle.State, dir puzzle.Direction, board
*puzzle.Board) (puzzle.State, int, bool) {
    currPos := startState.Pos
    nextNum := startState.NextNum
    cost := 0

    dRow, dCol := 0, 0
    switch dir {
    case puzzle.Up:
        dRow = -1
    case puzzle.Down:
        dRow = 1
    case puzzle.Left:
        dCol = -1
    case puzzle.Right:
        dCol = 1
    }

    moved := false

    for {
        nextPos := puzzle.Point{Row: currPos.Row + dRow, Col:
currPos.Col + dCol}

        // out of bounds
        if !puzzle.IsInBounds(nextPos, board) {
            return puzzle.State{}, 0, false
        }

        // hits wall
        if puzzle.IsWall(nextPos, board) {
            break
        }
    }
}

```

```

    }

    // continue sliding
    currPos = nextPos
    moved = true
    cost += puzzle.GetCost(currPos, board)

    // lava
    if puzzle.IsLava(currPos, board) {
        return puzzle.State{}, 0, false
    }

    // checkpoint
    for num, pt := range board.Checkpoint {
        if currPos == pt {
            if num == nextNum {
                nextNum++
            } else if num > nextNum { // not sequential
                return puzzle.State{}, 0, false
            }
            break
        }
    }

    if puzzle.IsGoal(currPos, board) {
        nextCell := puzzle.Point{Row: currPos.Row + dRow,
            Col: currPos.Col + dCol}
        isBlocked := !puzzle.IsInBounds(nextCell, board) ||
            puzzle.IsWall(nextCell, board)

        // stop di 0 (kalau belum semua checkpoint, nanti
        lanjut lagi dengan loop di masing2 algo)
        if isBlocked {
            break;
        }
    }
}

newState := puzzle.State{
    Pos:      currPos,
    NextNum:  nextNum,
}

```

```
    return newState, cost, moved
}
```

priorityQueue.go

```
package puzzle

type Direction string

const (
    Up    Direction = "UP"
    Down  Direction = "DOWN"
    Left  Direction = "LEFT"
    Right Direction = "RIGHT"
    Nil   Direction = "NIL"
)

type Node struct {
    State  State
    Cost   int
    Depth  int
    Parent *Node
    Dir    Direction
}

type State struct {
    Pos      Point
    NextNum  int
}

type PriorityQueue struct {
    Nodes []*Node
}

// add new nodes and make an adjustment
func (pq *PriorityQueue) Push(n *Node) {
    pq.Nodes = append(pq.Nodes, n)
    pq.upHeap(len(pq.Nodes) - 1)
}

// take and remove nodes with min cost
```

```

func (pq *PriorityQueue) Pop() *Node {
    if len(pq.Nodes) == 0 {
        return nil
    }

    min := pq.Nodes[0]

    // put last element to the root
    lastIdx := len(pq.Nodes) - 1
    pq.Nodes[0] = pq.Nodes[lastIdx]
    pq.Nodes = pq.Nodes[:lastIdx]

    // adjust the heap
    if len(pq.Nodes) > 0 {
        pq.downHeap(0)
    }

    return min
}

// check empty
func (pq *PriorityQueue) IsEmpty() bool {
    return len(pq.Nodes) == 0
}

// pick up node with smallest cost and adjust the heap
func (pq *PriorityQueue) upHeap(index int) {
    for index > 0 {
        parent := (index - 1) / 2

        // child >= parent
        if pq.Nodes[index].Cost >= pq.Nodes[parent].Cost {
            break
        }

        // switch pos with parent
        pq.Nodes[index], pq.Nodes[parent] = pq.Nodes[parent],
        pq.Nodes[index]
        index = parent
    }
}

```

```
// put down node with larger cost and adjust the heap
func (pq *PriorityQueue) downHeap(index int) {
    lastIdx := len(pq.Nodes) - 1
    for {
        left := 2*index + 1
        right := 2*index + 2
        smallest := index

        // check left child
        if left <= lastIdx && pq.Nodes[left].Cost <
            pq.Nodes[smallest].Cost {
            smallest = left
        }

        // check right child
        if right <= lastIdx && pq.Nodes[right].Cost <
            pq.Nodes[smallest].Cost {
            smallest = right
        }

        if smallest == index {
            break
        }

        // switch pos with smallest
        pq.Nodes[index], pq.Nodes[smallest] =
            pq.Nodes[smallest], pq.Nodes[index]
        index = smallest
    }
}
```

heuristic.go (hanya 1)

```
package puzzle

func CalculateHeuristic(board *Board, state State) int {
    var target Point
    if state.NextNum < len(board.Checkpoint) {
        target = board.Checkpoint[state.NextNum]
    } else {
        target = board.Goal
    }
}
```

```

    dr := state.Pos.Row - target.Row
    if dr < 0 {
        dr = -dr
    }
    dc := state.Pos.Col - target.Col
    if dc < 0 {
        dc = -dc
    }
    return dr + dc
}

```

parser.go

```

package filehandler

import (
    "bufio"
    "fmt"
    "iceSlidingPuzzle/src/puzzle"
    "os"
    "strconv"
    "strings"
)

func ParseBoard(filename string) (*puzzle.Board, error) {
    file, err := os.Open(filename)
    if err != nil {
        return nil, fmt.Errorf("Failed to open file: %w", err)
    }
    defer file.Close()

    scanner := bufio.NewScanner(file)

    // Read first line to get N and M
    if !scanner.Scan() {
        return nil, fmt.Errorf("Empty file / Failed to read first line")
    }
    firstLine := strings.TrimSpace(scanner.Text())

    // read N and M from first line
    var N, M int

```

```

_, err = fmt.Sscanf(firstLine, "%d %d", &N, &M)
if err != nil {
    return nil, fmt.Errorf("Failed to parse N and M: %w",
        err)
}

// Init board
board := &puzzle.Board{
    N:      N,
    M:      M,
    Grid:   make([][]rune, N),
    Cost:   make([][]int, N),
    Checkpoint: make(map[int]puzzle.Point),
    Lava:   []puzzle.Point{},
}

// read the grid lines
for row := 0; row < N; row++ {
    if !scanner.Scan() {
        return nil, fmt.Errorf("Expected %d lines but got
            fewer", N)
    }
    line := strings.TrimSpace(scanner.Text()) // remove
        space

    // validate line length
    if len(line) != M {
        return nil, fmt.Errorf("Invalid board: row %d has %d
            columns instead of %d", row+1, len(line), M)
    }

    board.Grid[row] = make([]rune, M)
    for col, char := range line {
        board.Grid[row][col] = char
        switch char {
            case 'Z':
                board.Start = puzzle.Point{Row: row, Col: col}
            case 'O':
                board.Goal = puzzle.Point{Row: row, Col: col}
            case 'L':
                board.Lava = append(board.Lava,
                    puzzle.Point{Row: row, Col: col})
        }
    }
}

```

```

        case '0', '1', '2', '3', '4', '5', '6', '7', '8',
            '9':
            board.Checkpoint[int(char-'0')] =
                puzzle.Point{Row: row, Col: col}
        }
    }

// read the cost lines
for row := 0; row < N; row++ {
    if !scanner.Scan() {
        return nil, fmt.Errorf("Expected %d lines but got
            fewer", N)
    }
    line := strings.TrimSpace(scanner.Text()) // remove
        space

    fields := strings.Fields(line) // parse multiple digit
        number

    // validate line length
    if len(fields) != M {
        return nil, fmt.Errorf("Invalid cost board: row %d
            has %d columns instead of %d", row+1, len(fields),
            M)
    }

    board.Cost[row] = make([]int, M)
    for col, nums := range fields {
        cost, err := strconv.Atoi(nums)
        if err != nil {
            return nil, fmt.Errorf("Invalid cost value at
                row %d, col %d: %s", row+1, col+1, nums)
        }
        board.Cost[row][col] = cost
    }
}

if err := scanner.Err(); err != nil {
    return nil, fmt.Errorf("Error reading file: %w", err)
}

```

```
    return board, nil
}
```

output.go (export output)

```
package filehandler

import (
    "fmt"
    "os"
    "strings"

    "iceSlidingPuzzle/src/puzzle"
)

func getDirectionStr(from, to puzzle.Point) string {
    if to.Row < from.Row {
        return "U"
    } else if to.Row > from.Row {
        return "D"
    } else if to.Col < from.Col {
        return "L"
    } else if to.Col > from.Col {
        return "R"
    }
    return "?"
}

func getBoardStateStr(board *puzzle.Board, currentPos
puzzle.Point) string {
    var sb strings.Builder
    for r := 0; r < board.N; r++ {
        for c := 0; c < board.M; c++ {
            if r == currentPos.Row && c == currentPos.Col {
                sb.WriteString("Z")
            } else if r == board.Start.Row && c ==
board.Start.Col {
                sb.WriteString("*")
            } else {
                sb.WriteRune(board.Grid[r][c])
            }
        }
    }
}
```

```

        sb.WriteString("\n")
    }
    return sb.String()
}

func SaveOutputTxt(board *puzzle.Board, pathTaken
[]puzzle.Point, cost int, totalNodes int, executionTime float64,
filename string) error {
    file, err := os.Create(filename)
    if err != nil {
        return err
    }
    defer file.Close()

    var steps []string
    for i := 1; i < len(pathTaken); i++ {
        steps = append(steps, getDirectionStr(pathTaken[i-1],
            pathTaken[i]))
    }
    file.WriteString(fmt.Sprintf("Solusi gerakan yang ditemukan:
%s\n", strings.Join(steps, ", ")))
    file.WriteString(fmt.Sprintf("Cost dari solusi: %d\n",
cost))

    if len(pathTaken) > 0 {
        file.WriteString("Initial\n")
        file.WriteString(getBoardStateStr(board, pathTaken[0]))
        file.WriteString("\n")
    }

    for i := 1; i < len(pathTaken); i++ {
        dir := getDirectionStr(pathTaken[i-1], pathTaken[i])
        file.WriteString(fmt.Sprintf("Step %d : %s\n", i, dir))
        file.WriteString(getBoardStateStr(board, pathTaken[i]))
        file.WriteString("\n")
    }
    file.WriteString(fmt.Sprintf("Waktu eksekusi: %.3f ms\n",
executionTime))
    file.WriteString(fmt.Sprintf("Banyak iterasi yang dilakukan:
%d iterasi\n\n", totalNodes))
    return nil
}

```

main.go (banyak panggil dari folder gui)

```
package main

import (
    "iceSlidingPuzzle/src/gui"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/app"
)

func main() {
    a := app.NewWithID("com.Arctic")
    a.Settings().SetTheme(&gui.ArcticTheme{})

    w := a.NewWindow("Arctic Solver")
    w.Resize(fyne.NewSize(1280, 720))
    w.SetFixedSize(false)

    mainUI := gui.NewMainUI(a, w)
    w.SetContent(mainUI.Build())

    w.ShowAndRun()
}
```

3. Implementasi GUI

GUI dibangun menggunakan *fyne*, yaitu framework GUI untuk bahasa Golang.

Terdapat 3 *page* utama yang dibuat pada GUI ini, yaitu *libraryPage*, *solverPage*, dan *resultPage*.

- **libraryPage**: menampilkan *history* puzzle (bisa duplikat meski nama *file* sama), menyediakan opsi memilih atau *load* puzzle, dan menyediakan fitur mencari *file* berdasarkan namanya. Bertanggung jawab membaca file input dan menginisialisasi board yang akan dipakai. Selain itu, memberikan keterangan kesulitan board (*extra*), **easy** (jumlah baris atau kolom (N/M) ≤ 7), **medium** (jumlah baris atau kolom (N/M) $> 7 \ \&\& \leq 12$), **hard** (jumlah baris atau kolom (N/M) > 12).

```
if maxDim <= 7 {
```

```

        diff = Easy
    } else if maxDim <= 12 {
        diff = Intermediate
    } else {
        diff = Hard
    }

```

```

func difficultyBadge(label string, style badgeStyle)
fyne.CanvasObject {
    bg := canvas.NewRectangle(style.bg)
    bg.CornerRadius = 3
    bg.StrokeColor = style.border
    bg.StrokeWidth = 1

    text := canvas.NewText(label, style.fg)
    text.TextSize = 9
    text.TextStyle = fyne.TextStyle{Bold: true}

    return container.NewStack(bg,
container.NewPadded(text))
}

func easyBadge() fyne.CanvasObject {
    return difficultyBadge("EASY", badgeStyle{
        bg:      ColorGreenBg,
        fg:      ColorGreenText,
        border: ColorGreenBorder,
    })
}

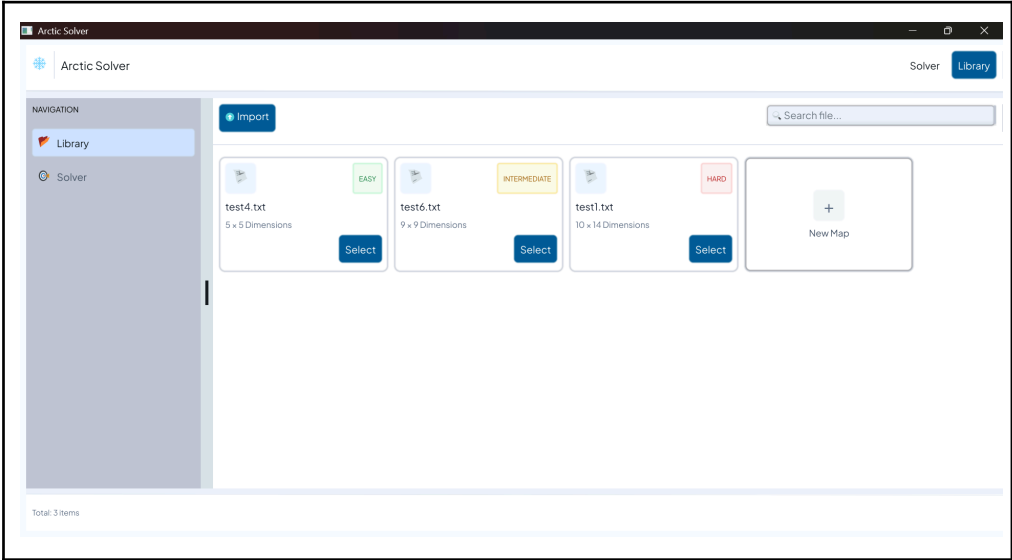
func intermediateBadge() fyne.CanvasObject {
    return difficultyBadge("INTERMEDIATE", badgeStyle{
        bg:      ColorYellowBg,
        fg:      ColorYellowText,
        border: ColorYellowBorder,
    })
}

func hardBadge() fyne.CanvasObject {
    return difficultyBadge("HARD", badgeStyle{
        bg:      ColorRedBg,

```

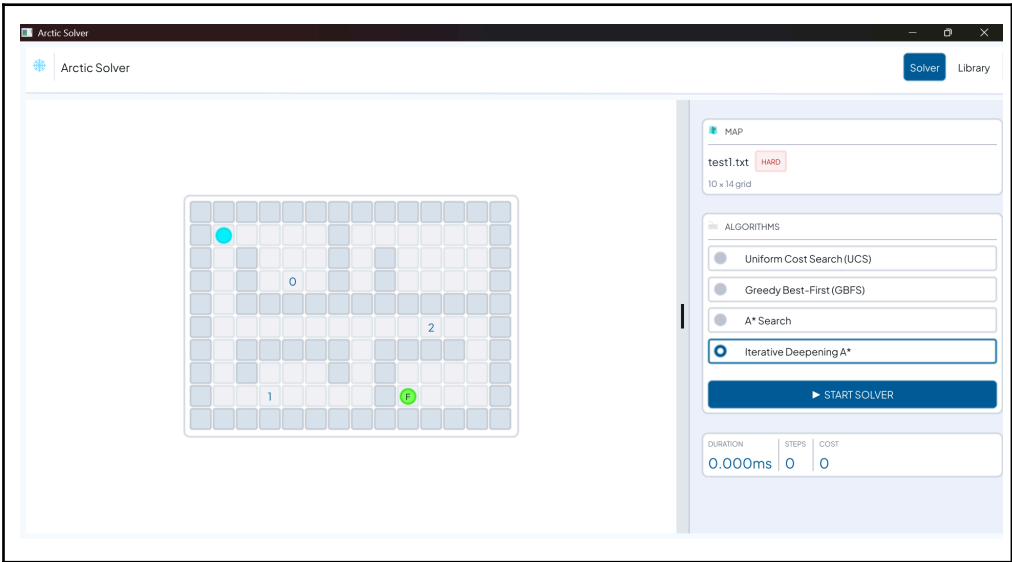
```
fg:    ColorRedText,
border: ColorRedBorder,
    })
}
```

Tampilan libraryPage



- **solverPage:** UI untuk memilih algoritma (A*, GBFS, UCS, IDA*), memberikan visualisasi board, dan menyediakan *button* solve.

Tampilan solverPage (*stats masih 0 karena belum di-run*)



- **resultPage:** Menampilkan animasi rute sekali (*setelah memencet tombol start solver*), statistik (cost, total moves, waktu), opsi ekspor, langkah - langkah rutanya. Selain itu, *page* juga menyediakan opsi *playback* (maju, mundur, pause, play)

Tampilan resultPage

The screenshot displays the 'Arctic Solver' application interface. The main window is titled 'Arctic Solver' and features a 'Visualizer' section on the left and an 'Execution Log' section on the right. The visualizer shows a 10x10 grid with a blue path indicating the solution. A green banner at the top of the visualizer area reads '✓ SOLUTION FOUND'. Below the grid, there are navigation controls: a play button, a reset button, and a step forward button. At the bottom of the visualizer, a summary table provides the following data:

TOTAL MOVES	TOTAL COST	TOTAL TIME
11 Steps	179 Credits	1.773 ms

The 'Execution Log' section on the right contains a table with the following data:

STEP	ACTION
01	Move Right 4 tiles
02	Move Down 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Left 2 tiles
06	Move Down 7 tiles

At the bottom right of the application, there are two buttons: 'SAVE OUTPUT' and 'BACK TO DASHBOARD'.

BAB 3

UJI PROGRAM

1. Kasus Uji 1 - *valid*

Input :

10 14

```
XXXXXXXXXXXXXXXXX
XZ*****X*****X
X*X***X*X***X
X*X*0*X*X***X
X*XXXXXXXXXXXXX
X*****2**X
X*XXXXX*XXXXX*X
X*X***X*X***X
X**1****XO***X
XXXXXXXXXXXXXXXXX
```

```
999 999 999 999 999 999 999 999 999 999 999 999 999 999
999 2 4 1 3 2 999 3 6 1 4 2 3 999
999 3 999 4 5 6 999 8 9 6 7 7 8 999
999 4 999 5 6 7 8 9 1 2 999 8 9 999
999 5 999 999 999 999 999 999 999 999 999 999 999
999 6 7 8 9 1 2 3 4 5 6 7 8 999
999 7 999 999 999 999 999 5 999 999 999 999 3 999
999 8 999 1 2 3 4 5 6 7 8 999 3 999
999 9 2 3 4 5 6 7 8 9 1 2 4 999
999 999 999 999 999 999 999 999 999 999 999 999 999
```

Output (UCS) :

Arctic Solver

Solver Library

Visualizer

✓ SOLUTION FOUND

Execution Log

STEP	ACTION
01	Move Right 4 tiles
02	Move Down 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Left 2 tiles
06	Move Down 7 tiles

TOTAL MOVES: 11 Steps
TOTAL COST: 179 Credits
TOTAL TIME: 0.629 ms

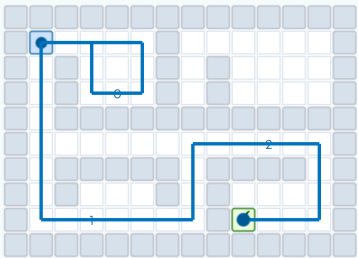
SAVE OUTPUT BACK TO DASHBOARD

Output (GBFS) :

Arctic Solver

Solver Library

Visualizer ✓ SOLUTION FOUND



Execution Log

STEP	ACTION
01	Move Right 4 tiles
02	Move Down 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Left 2 tiles
06	Move Down 7 tiles

TOTAL MOVES: 11 Steps
TOTAL COST: 179 Credits
TOTAL TIME: 0.526 ms

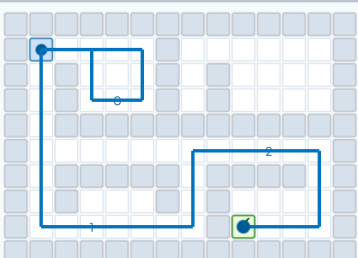
SAVE OUTPUT BACK TO DASHBOARD

Output (A*) :

Arctic Solver

Solver Library

Visualizer ✓ SOLUTION FOUND



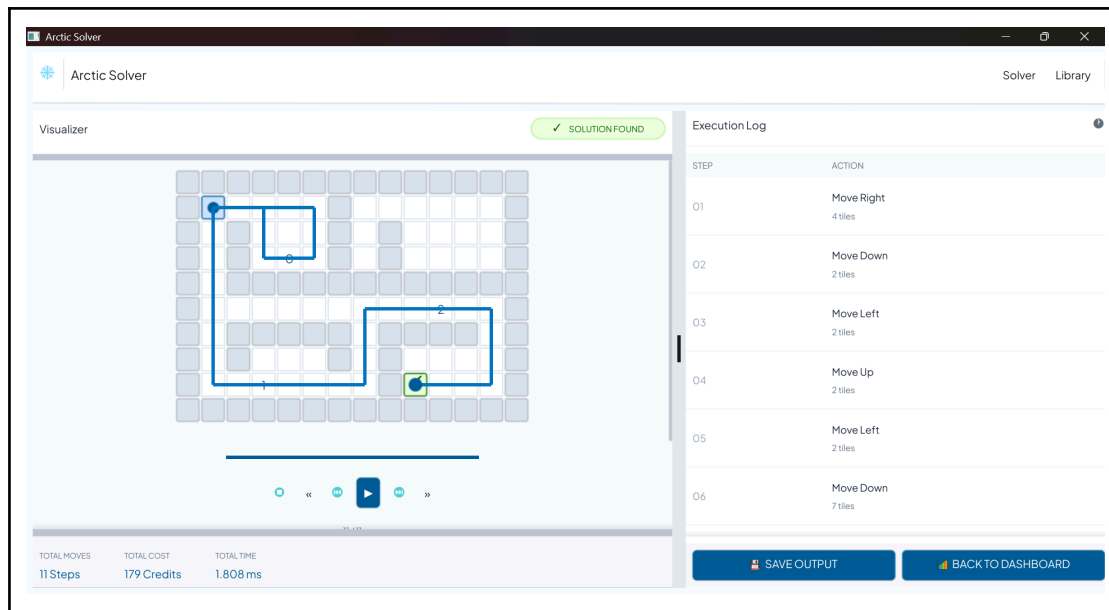
Execution Log

STEP	ACTION
01	Move Right 4 tiles
02	Move Down 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Left 2 tiles
06	Move Down 7 tiles

TOTAL MOVES: 11 Steps
TOTAL COST: 179 Credits
TOTAL TIME: 0.761 ms

SAVE OUTPUT BACK TO DASHBOARD

Output (IDA*) :



2. Kasus Uji 2 - *invalid (no N and M)*

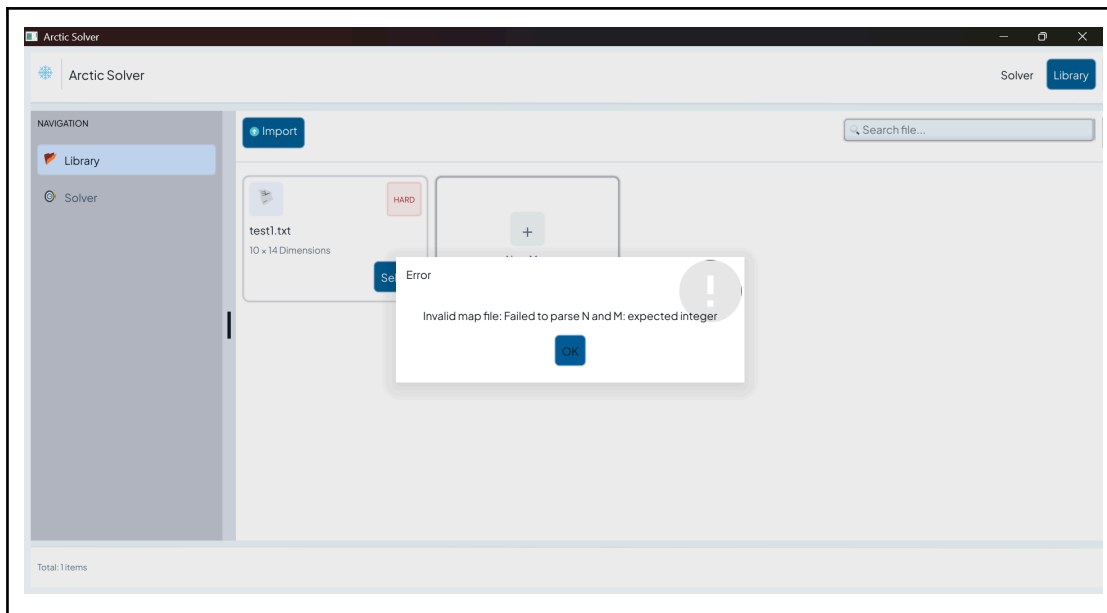
Input :

```

XXXXXXXX
X0****X
X**X**X
X***OX
X***1LX
XZ**X*X
XXXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999

```

Output :



3. Kasus Uji 3 - *invalid (suatu baris jumlah kolomnya $\neq M$)*

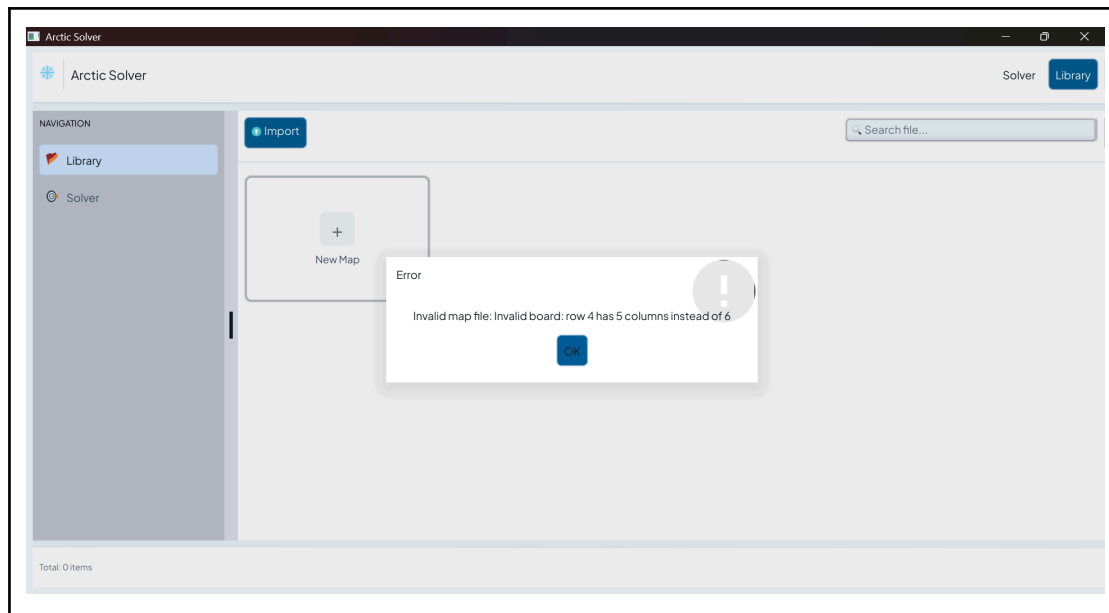
Input :

```

6 6
XXXXXX
XZ***X
X*X**X
X***X
X*0*OX
XXXXXX
999 999 999 999 999 999
999 1   2   3   4   999
999 5   999 6   7   999
999 8   2   9   1   999
999 3   4   5   6   999
999 999 999 999 999 999

```

Output :



4. Kasus Uji 4 - *valid*

Input :

```

7 7
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999

```

Output(UCS) :

Arctic Solver

Solver Library

Visualizer

✓ SOLUTION FOUND

TOTAL MOVES
8 Steps

TOTAL COST
87 Credits

TOTAL TIME
0.701 ms

Execution Log

STEP	ACTION
01	Move Right 2 tiles
02	Move Up 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Down 4 tiles
06	Move Right 2 tiles

SAVE OUTPUT
BACK TO DASHBOARD

Output (GBFS) :

Arctic Solver

Solver Library

Visualizer

✓ SOLUTION FOUND

TOTAL MOVES
8 Steps

TOTAL COST
87 Credits

TOTAL TIME
0.558 ms

Execution Log

STEP	ACTION
01	Move Right 2 tiles
02	Move Up 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Down 4 tiles
06	Move Right 2 tiles

SAVE OUTPUT
BACK TO DASHBOARD

Output (A*) :

Arctic Solver

Solver Library

Visualizer

✓ SOLUTION FOUND

Execution Log

STEP	ACTION
01	Move Right 2 tiles
02	Move Up 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Down 4 tiles
06	Move Right 2 tiles

TOTAL MOVES: 8 Steps
TOTAL COST: 87 Credits
TOTAL TIME: 1.190 ms

SAVE OUTPUT BACK TO DASHBOARD

Output (IDA*) :

Arctic Solver

Solver Library

Visualizer

✓ SOLUTION FOUND

Execution Log

STEP	ACTION
01	Move Right 2 tiles
02	Move Up 2 tiles
03	Move Left 2 tiles
04	Move Up 2 tiles
05	Move Down 4 tiles
06	Move Right 2 tiles

TOTAL MOVES: 8 Steps
TOTAL COST: 87 Credits
TOTAL TIME: 1.304 ms

SAVE OUTPUT BACK TO DASHBOARD

5. Kasus Uji 5 - No solution (lewati lava)

Input :

```

9 9
XXXXXXXXXX
XZ*****X
X*XXX***X
X*X*0***X
X*X2XXXXX
X*1*X***X

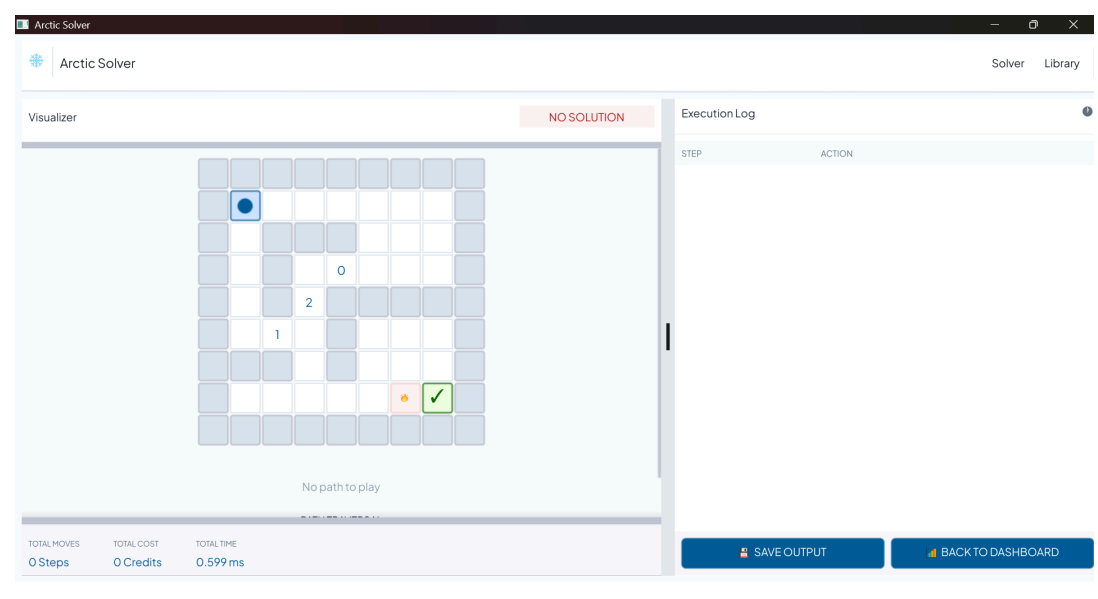
```

```

XXX*X**X
X*****LOX
XXXXXXXXXX
999 999 999 999 999 999 999 999 999
999 2   4   1   3   2   5   3   999
999 6   999 999 999 4   7   2   999
999 3   999 8   1   5   3   6   999
999 5   999 4   999 999 999 999 999
999 1   7   5   999 3   1   4   999
999 999 999 3   999 5   8   2   999
999 2   3   6   1   4   2   5   999
999 999 999 999 999 999 999 999 999

```

Output :



6. Kasus Uji 6 - valid

Input :

```

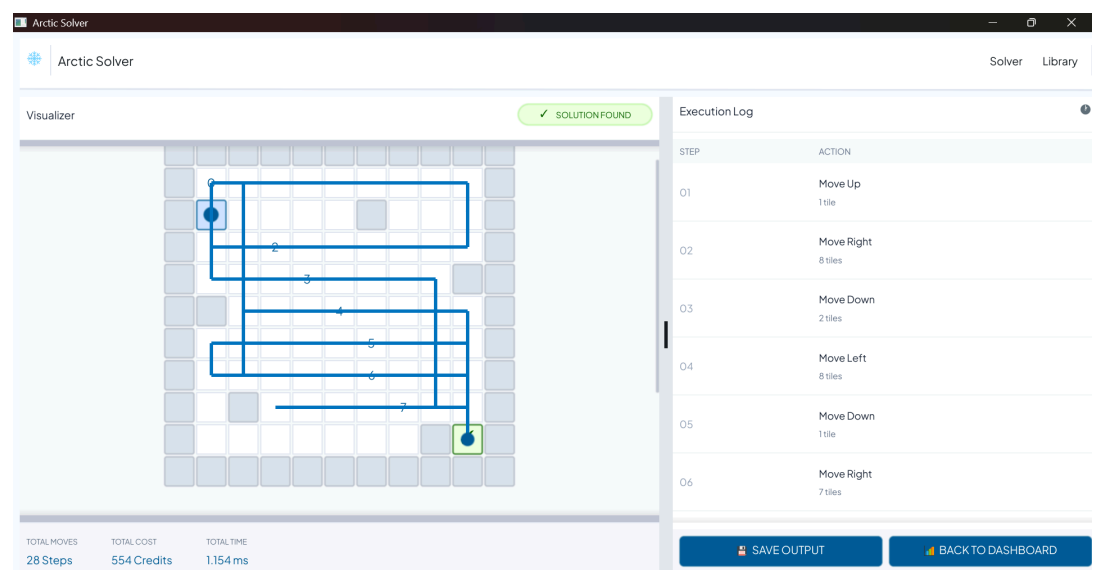
11 11
XXXXXXXXXXXX
X0*****X
XZ*****X*1X
X**2*****X
X***3*****XX
XX***4*****X
X*****5***X
X*****6***X
X*X*****7**X
X*****XOX

```

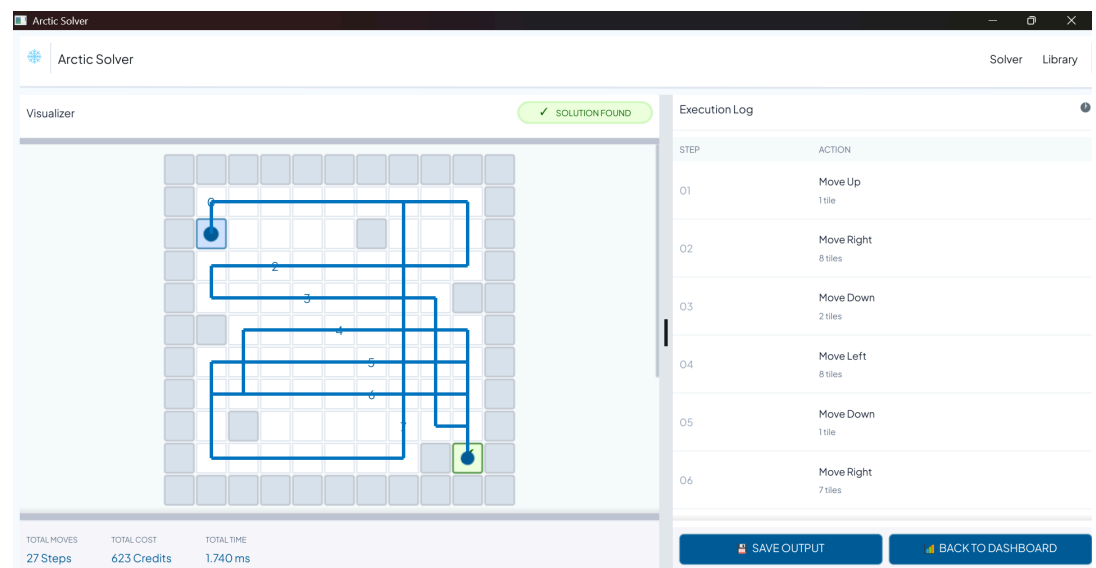
XXXXXXXXXX

```
999 999 999 999 999 999 999 999 999 999 999
999  7  3  5  2  8  4  6  1  9 999
999  4  6  2  9  1 999  3  8  5 999
999  2  8  4  6  3  7  5  9  1 999
999  5  1  9  3  7  2  8  4  6 999
999 999  3  6  8  4  9  1  7  5 999
999  8  4  7  1  6  3  9  2  4 999
999  3  9  1  5  2  8  4  6  7 999
999  6 999  5  3  9  1  7  2  8 999
999  1  4  8  7  5  6  2 999  3 999
999 999 999 999 999 999 999 999 999 999
```

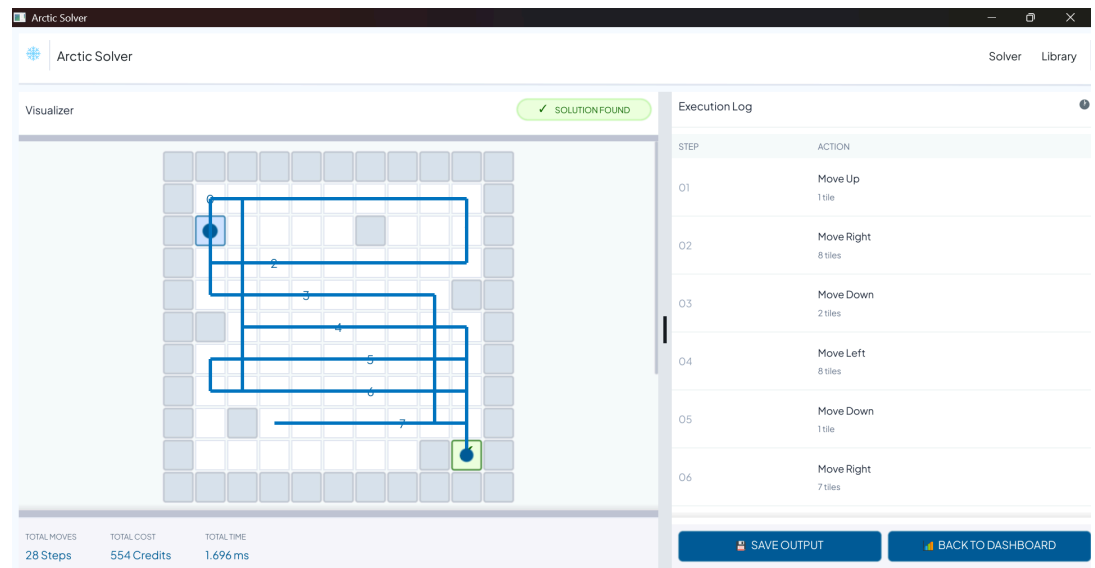
Output(UCS) :



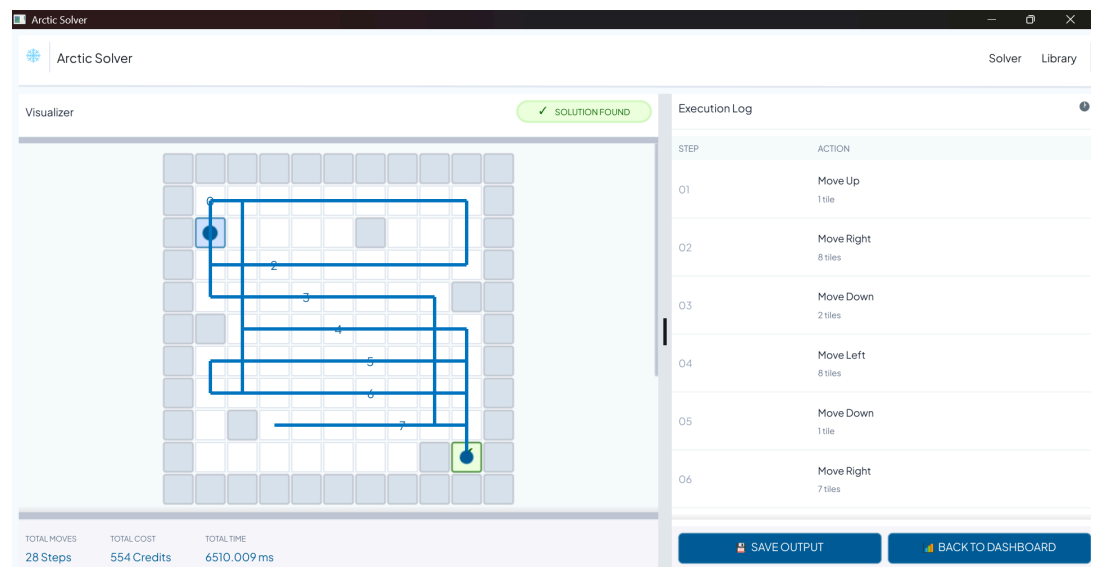
Output (GBFS) :



Output (A*) :



Output (IDA*) :



7. Analisis Hasil Percobaan Algoritma

Aspek	Best Algorithm	Keterangan
Cost Optimal	UCS / A* / IDA*	Selalu menghasilkan solusi dengan biaya minimum
Minimal Steps	GBFS	Hanya fokus pada node yang dekat dengan tujuan. Terbukti pada kasus 6,

		GBFS hanya membutuhkan <i>27 steps</i> , sedangkan algoritma lainnya membutuhkan <i>28 steps</i>
Waktu Tercepat	GBFS	Bersifat <i>greedy</i> karena hanya fokus pada node yang terlihat paling dekat dengan tujuan berdasarkan fungsi heuristik ($f(n) = h(n)$) sehingga dapat mencapai tujuan dengan waktu yang lebih singkat.

Tidak ada satu algoritma yang unggul dalam segala aspek. Pemilihan algoritma harus disesuaikan dengan prioritas utama yang ingin dicapai

BAB IV

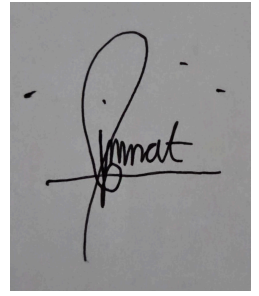
LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Valentino Daniel Kusumo

A handwritten signature in black ink on a light gray background. The signature is stylized, starting with a large loop and ending with a horizontal line. The word "Nathan" is partially visible.

Nathan Adhika Santosa